



SYSTEM REFERENCE

DESIGN / REQUIREMENTS / VERIFICATION

Technical specifications.

A system for acquiring data.
An architecture for preserving evidence.

PUBLISHED / UTC
2026-10-01

COLLECTION
41 specifications

IMPLEMENTATION
Status is recorded in each specification.

THE COLLECTION

Contents

Demo edition: full content for the first 5 specifications only.
All specifications are listed. A dash marks content omitted from this demo.
Select an included section to navigate. Numbers refer to pages in the companion PDF.

SPECIFICATION / SECTION	PAGE
01 Specification: acquisition context and custody references	16
Outcome and boundaries	16
Session artifact and binding	16
Context declaration command contract	17
Tool and environment fields	18
Validation report JSON contract	19
Custody metadata	21
External custody attestation	22
Acceptance	22
02 Specification: acquisition evaluation infrastructure	24
Outcome and scope	25
1. Streaming 8 GiB fixture pass (for E02)	25
2. Process-kill injection harness (for E05)	26
3. Controller failpoints (for E06)	27
4. One-million-row queue generator and resource sampler (for E10)	29
5. Concurrency-timing assertions (for E13)	30
6. Five-item scoped run assertion (for E11)	30
Deliverables	31
Next steps	32

03	Specification: acquisition fault-recovery tests	33
	Test boundary	33
	Required recovery cases	33
	Transfer and scheduler cases	34
	Assertions and artifacts	34
	Acceptance criteria	35

04	Specification: multi-route acquisition	36
	Outcome and scope	36
	Route resolution	36
	Engine integration points	36
	Schema and provenance	37
	Mixed-route startup gate	37
	Acceptance and next steps	38
	Native route selection	38

05	Specification: acquisition provenance manifest	39
	Purpose and boundary	39
	Record set	39
	Event format	40
	Acquisition fields	41
	Summary and verification	43
	Acceptance criteria	43

06	Specification: acquisition tool evaluation	—
	Outcome and scope	—
	Evaluation protocol	—
	Candidates and decision order	—
	Tor and process model	—
	Evaluation fixtures	—
	Deliverables and selection gate	—

Next steps	-
Native candidate	-

07 Specification: console activity log -

One event, one line	-
Concise messages	-
Acceptance criteria	-

08 Specification: console detail layout -

Header	-
Two-column field grid	-
Unknown values	-
Value formats	-
Secondary telemetry	-
Acceptance criteria	-

09 Specification: console Errors / review view -

Scope and data	-
Row content	-
Error categories	-
Retry eligibility	-
Details	-
Interaction	-
Visual style	-
Empty and failure states	-
Acceptance criteria	-

10 Specification: read-only console inspection -

Authority and transport	-
Protocol and records	-
Bounds and refresh	-

Privacy and unavailable data	-
Acceptance criteria	-

11 Specification: console item details	-
Entry and identity	-
Required sections	-
Source reveal and navigation	-
Refresh and controls	-
Acceptance criteria	-

12 Specification: console keymap	-
One key, one meaning	-
Focus regions	-
Footer	-
Help screen	-
Acceptance criteria	-

13 Specification: console queue view	-
Scope and layout	-
Filters and literal search	-
Pagination and consistency	-
Visual style	-
Interaction and retry information	-
Acceptance criteria	-

14 Specification: acquisition console UI	-
Outcome and dependencies	-
Launch and lifecycle	-
Install and run the first release	-
Main screen	-
Time, trend, and storage status	-

Detail, queue, and activity views	-
Health and uncertainty	-
Layout and performance	-
Delivery and acceptance	-
Deferred controls and next steps	-

15 Specification: console visual style	-
Outcome and dependencies	-
Label and value rules	-
Emphasis rules	-
Status colors	-
Footer and modal text	-
Highlight and focus rules	-
Precedence	-
Acceptance criteria	-

16 Specification: console worker details	-
Entry and slot identity	-
Required information	-
Refresh and navigation	-
Acceptance criteria	-

17 Specification: console worker table	-
Stalled rows	-
Unknown sizes	-
Basename cell	-
Acceptance criteria	-

18 Specification: attached control launch	-
Outcome and dependencies	-
Terminal ownership	-

Launch sequence	-
Incompatible flags	-
Out of scope	-
Acceptance criteria	-

19 Specification: controller command channel and interactive UI -

Outcome and authority boundary	-
Local command transport and authentication	-
Command set and confirmation	-
Textual application modes and update loop	-
Failure handling and recovery	-
Verification and delivery	-
Next steps	-

20 Specification: download telemetry and progress accounting -

Current implementation and integration boundary	-
Sources and authority	-
Snapshot transport and compatibility	-
Freshness, failure, and lifecycle	-
Phases and count reconciliation	-
Byte accounting and estimates	-
Speed and ETA calculations	-
Automatic safe-path remediation	-
Scope and eligibility	-
Mapping and promotion	-
Audit, telemetry, and recovery	-
Acceptance criteria	-
Verification and delivery	-
Next steps	-
Native measurements	-

21	Specification: qualified time and evidence preservation	-
	Outcome and separation of claims	-
	Time targets and policy	-
	Qualification verification	-
	Preservation package	-
	Archive acceptance and renewal	-
	Acceptance	-
22	Specification: external timestamps for custody checkpoints	-
	Purpose and limits	-
	Opt-in policy and service configuration	-
	Submission and immutable storage	-
	Verification and trust	-
	Failure and release behavior	-
	Acceptance and rollout	-
23	Specification: forensic compliance requirements and profiles	-
	Outcome and scope	-
	Requirement register	-
	Source corrections and unresolved research	-
	Technical profiles	-
	Assessment result	-
	Tool validation evidence	-
	Acceptance	-
24	Specification: HTTP and HTTPS acquisition transport	-
	Scope and changes to existing contracts	-
	Request descriptors and stable resource identity	-
	Descriptor controller integration	-
	Pre-contact provenance for descriptor attempts	-
	Descriptor retry-cooldown scheduling	-

Cooldown column	-
Scheduling loop	-
Retry requires a clean staging path	-
Configurable retry and backoff timing	-
Canonical source	-
CLI override	-
Active values	-
Resume	-
Out of scope	-
Descriptor status-code classification and access-denied pause	-
Carrying the status code	-
Classifying the failure	-
Origin pause	-
Source policy and route selection	-
Redirect handling	-
TLS policy and response observations	-
Engine-log and intercepted observation profiles	-
Descriptor observation profile	-
Body-capable descriptor transport	-
Recording a raised hop	-
Representation-safe resume	-
Descriptor items and resume_new_generation	-
Adapter and rollout requirements	-
Acceptance	-
Native URL-queue cutover	-

25 Specification: reproducible inventory discovery	-
Outcome and initial scope	-
Snapshot acquisition	-
Parsing and identity	-
Queue generation	-
Optional directory discovery	-
Acceptance and next steps	-

26	Specification: inventory rows to version 2 request descriptors	-
	Inputs and outputs	-
	Header record	-
	Request records	-
	Checks	-
	Mapping sidecar	-
	Changes to run custody	-
	Identity note	-
	Acceptance	-

27	Specification: inventory snapshot, manifest, and queue export	-
	Inputs and outputs	-
	Listing format	-
	Parse accounting and limits	-
	Snapshot store	-
	Identity and path mapping	-
	Policy file	-
	Manifest	-
	Queue export	-
	Snapshot diff	-
	Candidate manifest	-
	Policy validation against an earlier filter	-
	Acceptance	-
	Runbook	-
	Remaining work	-

28	Specification: manifest v2 evolution	-
	Outcome and ownership	-
	Reopened schema policy	-
	Digest and algorithm contracts	-
	Shared record rules	-
	Release and implementation order	-

Acceptance -

29 Specification: manifest and verification hardening -

Scope and compatibility -

Fail-closed verification -

Schema definitions and semantic checks -

 Inventory schema revisions -

Canonicalization and version transition -

Inventory validation before export -

Safe filesystem reads -

Resource bounds and error reporting -

Acceptance -

30 Native resume, cutover, and aria2 removal -

Terms -

Requirements -

Native resume contract -

 Eligibility and request checks -

 Authenticated range receipts -

 Generations, recovery, and uncertain bytes -

 Completion and verification -

Superseded statements -

Decisions -

Live signing repair -

Acceptance -

Rollout -

Review result and implementation gates -

31 Native transfer engine (release 1) -

Decision and principle -

What the aria2 engine cannot give -

The native engine -

Terms	-
Requirements	-
Superseded statements	-
Decisions	-
Acceptance	-
Rollout	-
Review result and implementation gates	-

32 Specification: reliable evidence acquisition -

Architecture and boundaries	-
Manifest and durable state	-
Selection and resource controls	-
Outage and retry behavior	-
Resume and representation integrity	-
Validation and finalization	-
Process, storage, and operator behavior	-
Migration and delivery order	-
Acceptance and next steps	-
Native cutover	-

33 Specification: response-body integrity observations -

Outcome and byte scope	-
Schema revision and compatibility	-
Profile selection and recording	-
Body-capable adapter contract	-
Descriptor items	-
Identity-version 1 native items	-
Observation contract	-
Staging binding	-
Final-file coverage and finalization	-
Recovery and review transitions	-
Admission and resource rules	-
Acceptance	-

Native authenticated ranges -

34 Specification: automatic resume script -

Outcome and dependencies -

When the script is written -

Flags -

Location and name -

Content -

Writing the file -

Out of scope -

Acceptance criteria -

Native cutover commands -

35 Specification: signed run custody and artifact lineage -

Guarantees and explicit boundaries -

Version 2 record envelope -

Selected items and pre-existing evidence -

Sessions, attempts, and recoveries -

Durable signed checkpoints -

Run index, outcomes, and rollback detection -

Inventory lineage -

 Inventory input locations -

 Lineage mode declaration -

Control decisions and handover -

Time and evidence after verification -

Command contract -

Acceptance -

Native cutover recovery -

36 Specification: multi-recipient sealed request evidence -

Outcome and ownership -

Chosen readings -

Terms	-
Multi-recipient sealing	-
Format 3	-
Schema fields and revision	-
Stop reasons	-
Boundaries	-
Security rules	-
Implementation order	-
Acceptance	-

37 Specification: authorized re-encryption of sealed request evidence	-
Outcome and ownership	-
Chosen readings	-
Terms	-
Authorized re-encryption	-
Authorization record	-
Derivation	-
Verification of a derived set	-
Boundaries	-
Security rules	-
Implementation order	-
Acceptance	-

38 Specification: sealed sidecar handover export	-
Outcome and ownership	-
Chosen readings	-
Terms	-
Handover policy	-
Export layout and signed rows	-
Verifying an included export	-
Boundaries	-
Security rules	-
Implementation order	-

Acceptance	-
------------------	---

39	Specification: on-demand target-directory rescan	-
-----------	---	----------

Outcome and initial scope	-
Manual trigger	-
File-system watch	-
Matching and checksum verification	-
Rescan report	-
Candidate queue generation	-
Acceptance and next steps	-

40	Specification: controller telemetry state projection	-
-----------	---	----------

Problem and scope	-
Required architecture	-
Projection contents and limits	-
Commit and update ordering	-
Startup, shutdown, and failure behavior	-
Concurrency and consistency	-
Tests and acceptance criteria	-
Next steps	-

41	Specification: Tor circuit recovery control	-
-----------	--	----------

Purpose and boundary	-
Authority and access	-
Command contract	-
Automatic recovery	-
Tor isolation and safety	-
Acceptance criteria	-

Specification: acquisition context and custody references

Status: implemented, September 25, 2026. The controller requires and validates a format 1 declaration before its first version 2 state write. It retains runtime context artifacts, binds them to signed records, and verifies the bindings and validation-report byte matches.

Outcome and boundaries

Bind each acquisition session to its actual tool configuration and declared custody context. Apply [v2 evolution](#) and [compliance](#). Preserve the existing separation between a signing key and a human identity.

The existing `tool_descriptor` role is an optional opaque run input. It cannot establish the environment of a later resumed session. Revised v2 sessions must use the structured context below. An old tool descriptor can remain a declared input, but cannot substitute for a session context.

Host measurements are collector assertions. Hashing an executable does not prove that the host was uncompromised or that those bytes executed. The report must state that boundary and name the validation evidence checked.

Session artifact and binding

Before `run_started` and source contact, write `context/<session-id>.json` below the run directory. Publish it durably with the existing no-overwrite rules. Add required `acquisition_context` artifact references to `run_started`, its session summary, and its run-index session entry. The references must agree and use run-relative paths.

An index entry for a session without an authenticated `run_started` has `acquisition_context: null`. Once that event exists, require its reference in every later index entry. No completed session can have a null context.

The artifact contains exactly `context_format: 1`, `run_id`, `session_id`, `captured_at`, `components`, `environment`, `effective_configuration`, `validation_report`, and `custody`. IDs and timestamps follow v2 types. The context has no independent signature; the signed event and index bind its exact bytes. The handover must copy it and verify those bindings.

The controller must regenerate context for every session. It must retain each prior context. A crash before the first signed event leaves an orphan artifact with no acquisition claim; recovery must not adopt it as a signed context for a different session. Existing incomplete-session rules apply.

Context declaration command contract

Each revised v2 session must receive one operator-owned JSON declaration:

```
--context-declaration PATH
```

PATH names a regular file that the controller opens without following a symbolic link. The controller must validate the declaration before it writes state or contacts a source. The controller must not retain, hash, export, or reference the declaration file itself. This keeps operator-only paths and any restricted handling information outside the acquisition package.

The declaration is UTF-8 JSON. It contains exactly these keys:

```
{
  "context_declaration_format": 1,
  "components": [
    {
      "component_id": "collector-1",
      "role": "collector",
      "name": "tod-dl",
      "version": "2.0.0",
      "artifact_path": "/secure/operator/tod-dl-source-manifest.json"
    }
  ],
  "redactions": ["aria2_rpc_secret"],
  "validation_report_path": "/secure/operator/validation-report.json",
  "validation_evidence_paths": [
    "/secure/operator/validation-evidence.json"
  ],
  "custody": {
    "case_id": "case-2026-001",
    "operator_ref": "operator-7",
    "organization_ref": "unit-3",
    "role_ref": "collector",
    "authority_ref": null,
  }
}
```

```

    "facility_ref": null,
    "custody_record_ref": "custody-2026-001-r1"
  }
}

```

`context_declaration_format` must equal `1`. Unknown keys fail validation. Each component object has exactly the keys shown above. Its identifier, role, name, and version populate the corresponding context component fields. `artifact_path` is an absolute operator input path. The controller must read the target as a regular file without following links, copy its exact bytes below the session `context/` tree, and use the resulting artifact reference. The component rules in "Tool and environment fields" still apply, including the required roles and sorted unique component identifiers.

`redactions` is a sorted array of unique secret parameter names. It can name parameters only; it must not contain a secret value, digest, credential, or command line. The controller records these names in the generated effective configuration artifact and omits the matching values.

`validation_report_path` names the one required validation report. Each entry in `validation_evidence_paths` names an additional report input. The evidence-path array must contain at least one entry for a matched report. The controller reads every path as a regular file without following links, copies and hashes each file below `context/`, and records safe artifact references. The generated validation report reference must bind the copied report. A declaration can use no duplicate path, including the validation report path.

`custody` has exactly the fields and value rules in "Custody metadata". `custody.case_id` must equal `--case-id`. The command must reject a declaration without `--case-id`, or with a different case ID, before any state write.

The controller captures environment and effective non-secret settings from the running session. The declaration does not override those measurements. It must reject invalid JSON, unsupported formats, unknown fields, duplicate component IDs or paths, unsafe files, unreadable files, and values that break the context artifact contract before it creates `context/` or `run_started`. Repeated component flags are not part of this interface.

Tool and environment fields

`components` is a nonempty array sorted by ascending ASCII byte order of `component_id`. Each entry has exactly `component_id`, `role`, `name`, `version`, and `artifact`. `component_id` is an opaque identifier matching `[A-Za-z0-9._-]{1,128}` and cannot equal `.` or `...`. `name` and `version` are nonempty Unicode scalar strings of at most 256 UTF-8 bytes, with no control characters. Role is `collector`, `runtime`, `adapter`, `transfer_engine`, `tor`, `routing_wrapper`, or `dependency`.

Require one collector, runtime, and adapter. Require every transfer engine and routing component actually used. Include the installed cryptographic, canonicalization, and validation dependencies. `artifact` refers to a retained file or deterministic source-package manifest under the context's run-relative `context/` tree. A source-package manifest lists relative paths, byte counts, and SHA-256 values, sorted by path, with no duplicates. It must include loaded source modules; a Git commit ID alone is insufficient.

Capture files from the configured execution paths, not an unrelated version command found on `PATH`. A profile must fail admission if a required component cannot be identified, hashed, or matched to validation evidence. Deployment procedures must prevent component replacement during a session. A detected component change must stop new admission and require a new session; it cannot rewrite the existing context.

`environment` has exactly `os_name`, `os_version`, `kernel_version`, `architecture`, `execution_kind`, `image_digest`, `host_ref`, and `clock`. Execution kind is `native`, `container`, or `virtual_machine`. Image digest is a nullable `digest_set`; it describes an image only when measured. `host_ref` is an opaque identifier. Do not collect raw hostnames, usernames, home paths, IP addresses, or a full environment-variable dump.

`clock` contains `source`, `synchronized`, `measured_at`, `offset_microseconds`, and `uncertainty_microseconds`. Source is `ntp`, `ptp`, `manual`, or `unknown`. Synchronization is Boolean or null when unavailable. Measurement time and numeric fields are nullable. Offset is a signed safe integer; uncertainty is nonnegative. Unmeasured values are null, never zero. Clock metadata is not external timestamp proof.

`effective_configuration` is an artifact reference to a canonical object containing exactly `configuration_format: 1`, `settings`, and `redactions`. Settings is a sorted array of unique `{name, value}` entries. A registered adapter schema defines the permitted settings and their value types. Include selection, validation, retry, resume, storage, route, and transport settings that affect acquisition. Represent paths through artifact or opaque references. Record secret parameter names in `redactions`; omit their values and hashes. Do not retain raw command lines that can contain credentials.

`validation_report` is a required artifact reference to a retained report under `context/`. Its configuration and component digests must match the current session under `compliance`. A report reference binds the report; it does not independently prove a reviewer performed its tests.

Validation report JSON contract

The declared report is UTF-8 JSON with exactly `validation_report_format`, `components`, `effective_configuration`, `evidence`, `fixtures`, `tests`, `known_limitations`, `unsupported_transport_features`, `review_ref`, and `reviewed_at`. `validation_report_format` is the integer `1`. Reject duplicate JSON keys, unknown keys, invalid UTF-8, and integers outside the v2 safe integer

range. Timestamps use the v2 timestamp type. The report and every evidence file contain no restricted text. Before the controller copies or hashes either input, it must decode the input as UTF-8 and reject it when the input contains restricted text. Evidence files must therefore be UTF-8 text.

For this contract, restricted text is any configured restricted value, an absolute POSIX path, a Windows drive or UNC path, a URI with user info, or a line that has an executable token followed by a whitespace-delimited option. The active profile supplies the restricted values outside the declaration and export. The values must include each known secret, hostname, username, and human identity. The controller compares values as exact Unicode scalar strings. It scans raw decoded text and, when an input is valid JSON, every decoded JSON string value. This rule does not require the controller to infer identities.

`components` is a nonempty array sorted by ascending ASCII byte order of `component_id`, with unique IDs. Each entry contains exactly `component_id`, `role`, `name`, `version`, `artifact_sha256`, and `artifact_byte_count`. The first four values have the same types and rules as context components. The digest is 64 lowercase hexadecimal characters. The byte count is a nonnegative safe integer.

`effective_configuration` contains exactly `sha256` and `byte_count`, with the same digest and count rules. The digest covers the exact retained bytes of the generated configuration artifact. It does not cover the operator declaration or a reconstructed settings object.

`evidence` is an array of objects with exactly `sha256` and `byte_count`, using the same digest and count rules. Entry `i` binds the exact bytes of `validation_evidence_paths[i]`, after the controller copies that file. The array length must equal the declared evidence-path count and be nonzero. The retained path for entry `i` is `context/<session-id>/validation-evidence/<i>`. The handover must copy each file at the same relative path.

`fixtures` is a nonempty array sorted by ascending ASCII byte order of unique `fixture_id`. Each entry contains exactly `fixture_id`, `sha256`, and `byte_count`. The digest and count identify the tested fixture bytes. A fixture digest is a report assertion; the controller does not fetch or reconstruct fixture bytes to check it. Run and handover verification checks fixture IDs and test references only. It reports fixture digests as unverified external assertions.

`tests` is a nonempty array sorted by ascending ASCII byte order of unique `test_id`. Each entry contains exactly `test_id`, `status`, `component_ids`, `evidence_indices`, and `fixture_ids`. Status is `pass`, `fail`, or `skip`. The three reference arrays are sorted and unique. Identifier arrays use ascending ASCII byte order. Evidence indices use ascending numeric order. `component_ids` and `fixture_ids` refer to entries in this report. Each evidence index is a nonnegative safe integer less than the length of `evidence`. A passing test has at least one component ID, evidence index, and fixture ID. Every component and evidence index must occur in at least one passing test. Every declared fixture must occur in at least one passing test's `fixture_ids`. Failed and skipped tests remain in the report; a failed test blocks admission.

`fixture_id`, `test_id`, and `review_ref` are opaque identifiers matching `[A-Za-z0-9._-]{1,128}`, excluding `.` and `...`. `known_limitations` is an array sorted by ascending ASCII byte order of unique `limitation_id`. Each entry contains exactly `limitation_id` and `summary`. The ID uses the same opaque-ID rule. Summary is a nonempty Unicode scalar string of at most 1,024 UTF-8 bytes. It has no control characters, raw absolute paths, command lines, credentials, or human identities. Reject a report with a summary that breaks these rules. `unsupported_transport_features` is an array sorted by ascending ASCII byte order of unique identifiers using the same opaque-ID rule. An empty array states that the report declares none. `review_ref` and `reviewed_at` identify the claimed review. They do not establish reviewer identity or accreditation.

Before `run_started` or source contact, the controller must compare the report's component count and each component's ID, role, name, version, artifact digest, and byte count with the collected session context. It must compare the configuration digest and byte count with the generated artifact. It must compare every evidence digest and byte count with the copied file at the same index. Missing, extra, reordered, malformed, or mismatched entries fail admission. A failed test or a component without a passing test also fails admission. Skips and unsupported features remain explicit for later profile assessment; an exact match alone does not establish validation sufficiency or legal compliance.

Run and handover verification must repeat the component, configuration, and evidence checks against retained bytes. It must repeat structural fixture and test checks: object keys and types, array order and uniqueness, references, index bounds, passing coverage, and failed-test rejection. It must report a failed match separately from a broken signed reference. Neither check executes tests or proves that a reviewer performed them.

Custody metadata

`custody` contains exactly `case_id`, `operator_ref`, `organization_ref`, `role_ref`, `authority_ref`, `facility_ref`, and `custody_record_ref`. Case ID must match both registries. The other values use opaque identifiers matching `[A-Za-z0-9._-]{1,128}`, excluding `.` and `...`. `authority_ref` and `facility_ref` are nullable; all other values are required. A profile requiring legal authority must reject a null authority reference.

These identifiers link to a restricted human custody record outside the run and handover export. That record holds actual identities, authority documents, evidence labels, transfer purpose, and transfer notes. An authority reference records a claim; the downloader cannot authorize access. Facility identity and route observations must not imply source geolocation.

Each human evidence label must map to the run ID and immutable item ID. Labels do not replace item IDs or determine output paths. Correct a label with a new custody-record revision; never change the signed acquisition. Record unavailable identities or authority information as explicit omissions in the external record. A profile requiring them must fail assessment.

External custody attestation

After handover, a separate signed attestation can bind a specific human custody-record revision to the released handover. Store it outside the export, like the existing recipient receipt. Do not change that receipt's acceptance semantics or rewrite the handover to insert the attestation.

The attestation contains exactly `record_type: custody_attestation`, `attestation_format: 1`, `run_id`, `handover_sha256`, `custody_record_ref`, `custody_revision_sha256`, `signer_ref`, `recorded_at`, `signing_key_fingerprint`, and `signature`. Use the v2 JCS signature rule. The custody revision digest binds the exact restricted document bytes. The signer reference is opaque; a key alone does not establish its identity.

A future verifier must accept explicit paths for the attestation, restricted revision, and externally trusted signer key when checking this linkage. It must verify signature, run and handover identity, record reference, and revision digest. It must never discover private records or fetch them. Missing private material gives `indeterminate` for the requested linkage check. Public handover integrity remains independently assessable.

Human identity and authority validation remain organizational procedures. Machine verification must report `signed_assertion`, the trust source, and whether the document linkage passed. It must not report verified legal authority solely because an attestation signature passed.

Acceptance

These gates test both accurate bindings and privacy boundaries.

- AC01: Two sessions with different environments retain distinct contexts; swapping either artifact fails signed-reference verification.
- AC02: A changed component or effective setting fails the validation match before source contact; a Git ID cannot hide modified source bytes.
- AC03: Secret-bearing flags, hostnames, and usernames never enter context, logs, configuration artifacts, or generated fixture reports.
- AC04: Unknown clock measurements remain null and cannot imply trusted time.
- AC05: Context case mismatch, duplicate component IDs, unknown fields, and unsafe paths fail without reading outside the declared artifact root.
- AC06: A custody attestation for another handover or document revision fails; missing restricted records do not become verified identity claims.
- AC07: Crash recovery preserves signed contexts and never attributes an orphan context to a later acquisition session.
- AC08: Malformed reports, changed component or configuration bytes, missing evidence, and failed tests block admission before source contact.

- AC09: Duplicate and unknown report keys, invalid references, incorrect ordering, unsafe text, and a fixture without a passing test block admission.
- AC10: Run and handover verification separately report a retained-byte match failure and a broken signed reference.

Specification: acquisition evaluation infrastructure

Status: implemented, September 18, 2026. All six pieces are built. Per [the second evaluation report](#), E11 had no harness at that report's writing; Section 6's harness (`e11_five_item_scoped_run` , `src/evaluation/run_acquisition_evaluation.py`) closes that gap, wired into `main()` , with a corresponding unit test for its new `_read_run_items` helper in `tests/test_acquisition_evaluation.py` . Per this specification's scope, E11 has not been run yet; running it and recording a result belongs to a later evaluation report. The five other built pieces: the streaming 8 GiB fixture pass for E02 (`Fixture` , `FixtureServer` , and `e02_large_file_interrupted_resume` , `src/evaluation/run_acquisition_evaluation.py`); the kill-injection harness for E05 (`start_downloader` , `src/evaluation/aria2_evaluation_adapter.py` ; `process_tree_pids` , `find_engine_pid` , `wait_for_bytes_written` , `src/evaluation/acquisition_evaluation.py` ; `e05_kill_injection`); the three named controller failpoints for E06 (`hit_failpoint` , `FAILPOINTS` , `src/tod_dl.py` , gated by `TOD_DL_FAILPOINT` ; `FAILPOINTS` now holds three more names, described in section 3) plus the `reconcile_promotions()` extension for failpoint (a), already built before this specification (`Downloader.reconcile_promotions()`) and `is_promotable_final_path()` , `src/tod_dl.py`); the one-million-row queue generator and resource sampler for E10 (`generate_synthetic_queue_rows` , `ResourceSampler` , `src/evaluation/acquisition_evaluation.py` ; `e10_million_row_admission_and_resources`); and the concurrency-timing assertions for E13 (`e13_concurrency_timing`), reading admission and completion timestamps from `download_transitions` via `read_transitions` (`src/evaluation/aria2_evaluation_adapter.py`). Per this specification's scope, none of E02, E05, E06, E10, or E13 was run as part of this work.

A later session wired E05, E06, E10, and E13 into `src/evaluation/run_acquisition_evaluation.py` 's `main()` and ran them, per `specs/reports/acquisition-tool-evaluation-2026-09-18b.md` : E10 and E13 pass; E05 and E06 fail on two real, confirmed controller gaps (a `scope_run()` / `reconcile_promotions()` ordering bug, plus a related stale `.aria2` -control-file false-failure in `Downloader.transfer()`), not infrastructure gaps — see that report's "Remaining gaps" section. E02 did not run to completion: its fixture generator sustained only ~3.4 MB/s in a partial run, too slow to finish within the 600s per-attempt time limit, confirming Section 1's unmeasured open question was a real cost, not a false worry. Separately, Section 1's `is_incomplete_body_failure()` gap is fixed outside this specification's scope: `Downloader.transfer()` (`src/tod_dl.py`) now retries a first "Got EOF from the server" failure through the normal `retry_wait` path instead of routing it straight to `review_required` , so a resumable mid-transfer cut can complete via `--continue=true` on the next attempt; it gives up to `review_required` only when a retry's incomplete-body

failure shows no growth in staged bytes, which is the signal for a genuinely short body. That fix is confirmed working for E02's own requirement (resumed past the 256 MiB interrupt point in the partial run above) independent of the generator's throughput problem.

A later session measured and fixed the generator throughput problem. `DeterministicBytes._block()` (`src/evaluation/acquisition_evaluation.py`) hashed a 64-byte `blake2b` digest per block, which needed 134 million hash calls to generate an 8 GiB fixture (~3.4 MB/s); separately, `Fixture.generator` returns a new `DeterministicBytes` instance on every access, so no per-instance block cache could ever hit. The fix switches to a `shake_256` digest producing 1 MiB blocks (8192 calls for an 8 GiB fixture) through a module-level `lru_cache` keyed on `(seed, block_index)`, which caches across `Fixture.generator` accesses. Measured: an 8 GiB fixture's `sha256()` pass now completes in ~16 s (>500 MB/s), well inside the 600 s per-attempt time limit. E02 has not yet been run to completion with the fixed generator.

Outcome and scope

Define the test infrastructure that [the acquisition tool evaluation specification](#) requires to run E02, E05, E06, E10, E11, and E13. This specification does not change any required result in that specification's scenario table. It does not evaluate a candidate and does not select a configuration. Build each piece as an addition to the existing harness (`src/evaluation/acquisition_evaluation.py`) and the existing adapter and runner (`src/evaluation/aria2_evaluation_adapter.py` , `src/evaluation/run_acquisition_evaluation.py`), not as a separate harness.

Implement the six pieces in the order listed below. Each piece stands alone: a later piece must not depend on an earlier piece beyond shared harness code that already exists.

1. Streaming 8 GiB fixture pass (for E02)

Requirement: "Generate fixture bytes from a documented seekable deterministic generator, so the server can serve an 8 GiB representation without materializing it first. The harness must calculate the expected digest with the same generator in a separate streaming pass." (parent specification, "Evaluation fixtures.")

- Add an 8 GiB fixture entry to the manifest generator, built from the existing deterministic generator (already used for smaller fixtures) run to a length of `8 * 1024**3` bytes. Confirm the existing generator is seekable at arbitrary byte offsets before reusing it; if it is not, that gap belongs in this section, not a new generator.
- Compute the expected SHA-256 in a second, independent streaming pass over the same generator and seed, never by hashing the served bytes. Record generator name, seed, length, and digest in the fixture manifest.
- Serve the fixture without materializing it: the fixture server's request handler must produce each response chunk from the generator at the requested byte offset, not from a file written to disk first.

- Interrupt the transfer after exactly 256 MiB, per E02's requirement, using the existing scripted-response mechanism (connection termination at a byte count). Assert on resume: a nonzero `Range` request, a final digest that matches the manifest, and no full restart from byte 0.
- Run this fixture once per viable candidate, per the parent specification's "Run the full large-file fixture once per viable candidate" instruction. Do not add it to the repeated failure-boundary test set.

Open question: confirm the current generator's per-chunk cost is low enough that an 8 GiB streamed response completes in a bounded time on the evaluation machine. Measure this during implementation; do not assume it.

Resolved: `terminate_after_bytes` (the scripted-response mechanism named above) closes the connection before the response reaches its advertised `Content-Length`, which aria2 reports as `GOT EOF FROM THE SERVER` — the same error text `is_incomplete_body_failure()` (`src/tod_dl.py`) matches to route a nonzero-size staging file to `review_required`, not `retry_wait`, since the E08/E12 fix in commit `a6ee1d9`. Commit `a6ee1d9`'s own docstring for `is_incomplete_body_failure()` confirms aria2 gives no distinguishable signal: it reports `errorCode=1` for this case, the same generic code it uses for unrelated failures, which is why that fix matches on message text rather than the code. aria2 has no way to tell a genuinely short body (the server will always send fewer bytes than `Content-Length`, which no retry fixes) apart from a connection cut mid-transfer that a `--continue=true` retry could resume. A transfer interrupted at 256 MiB therefore takes the `review_required` route under current controller code, which forecloses resume before a second `run_aria2()` attempt can ever try it, and fails E02's "resume, no full restart" requirement. This is a controller-side gap: `is_incomplete_body_failure()` cannot become more precise using only aria2's output, so the fix (if any) must live in what the controller does with a nonzero-size, `GOT EOF`-classified staging file, not in a more precise error match. Closing that gap is outside this specification's six pieces, per the scope line above ("This specification does not change any required result"); E02 cannot pass until a decision is made and implemented separately. Do not implement any change to `is_incomplete_body_failure()` or its caller as part of this specification.

2. Process-kill injection harness (for E05)

Requirement: "Kill the engine, controller, and both during transfer. Restart without losing jobs, concurrent duplicate writers, or premature promotion." (parent specification, scenario table.)

- Add a kill-injection mode driven from `src/evaluation/run_acquisition_evaluation.py`. The existing invocation, `run_downloader()` in `src/evaluation/aria2_evaluation_adapter.py`, uses `subprocess.run(...)`, which blocks until the controller exits and exposes no PID or live handle while it runs. Add a separate, `Popen`-based invocation function alongside `run_downloader()` in `src/evaluation/aria2_evaluation_adapter.py` — the module whose stated purpose is driving the controller subprocess — that starts the same controller command line and returns a live handle instead of a completed result. Do not reuse `run_downloader()` unmodified for this mode.

- For the engine sub-case, identify the aria2 process to signal by walking the process tree rooted at the controller's PID, the same approach Section 4 uses for RSS sampling: the controller spawns `torsocks -i aria2c ( Downloader.run_aria2() , src/tod_dl.py )`, and the actual `aria2c` process may be a direct child or a grandchild depending on whether `torsocks` exec-replaces itself on the evaluation machine. Confirm which during implementation before selecting a PID to kill; signaling the wrong process would leave the real writer running and invalidate the sub-case's "concurrent duplicate writer" assertion.
- Trigger the kill by byte-transferred count, measured by polling the size of the staging file the controller is writing to on disk (the destination and state directories are already known to the harness) rather than by wall-clock delay, so the kill point is reproducible across runs. The fixture server's event log cannot serve as the trigger: it records one event per request, written only in the request's `finally` block after the request completes or its connection is torn down (`FixtureServer._record` , `src/evaluation/acquisition_evaluation.py`), so it has no entry to observe during an in-progress transfer.
- After the kill, restart the controller against the same isolated destination and state directories used before the kill, and let it run to completion or its normal terminal state.
- Assert, from durable state and the final file: exactly one final file per item, and no promotion of a partial transfer that was in flight at kill time. The reliable acquisition specification does not enumerate valid partial-and-final coexistence states; the controller's own finalization path (`Downloader.transfer()` , `src/tod_dl.py`) defines the only legitimate coexistence window instead: the final file is created by `os.link(staging, target)` while `status="promoting"` , and staging is unlinked only after the following `status="complete"` transition commits. Assert that any staging-and-final coexistence observed after a kill and restart matches this window (a `promoting` or `complete` row for that URL at the moment of coexistence) and not a state outside it.
- Run three sub-cases: engine killed, controller killed, both killed (simultaneously, within the same event-log tick). Record which sub-case produced which durable state.

Open question: decide whether "both during transfer" means simultaneous `SIGKILL` to both processes or a kill to one shortly followed by a kill to the other before either can react. Pick simultaneous first, since it is the harder case; add the staggered case only if the evaluation report calls it out as a distinct gap.

3. Controller failpoints (for E06)

Requirement: "Test-only controller failpoints must stop immediately after durable validation intent, exclusive final-file creation, and completion-record commit. E06 then has unambiguous process boundaries." (parent specification, "Evaluation fixtures.")

- Add three named, test-only failpoints to the controller's finalization path (`Downloader.transfer()` , `src/tod_dl.py`), each stopping the process immediately after one step: (a) the durable record of validation intent — the `status="promoting"` transition, which is written only after hashing completes and already

carries the computed digest, immediately before the final file is created; (b) exclusive final-file creation (after `os.link(staging, target)` succeeds, before the completion record commits); and (c) the completion-record commit itself — the first `status="complete"` transition, which records `bytes` and `sha256` (after that database write, before the staging file is unlinked and the second, cleanup `status="complete"` transition that follows it).

- Gate each failpoint behind an environment variable or CLI flag that only the evaluation runner sets, never a flag reachable in normal operation. Name the three failpoints distinctly (for example, `TOD_DL_FAILPOINT=post_validation_intent`, `post_final_file_creation`, `post_completion_commit`) so a single run exercises exactly one boundary.
- E06 needs exactly these three. `FAILPOINTS` also holds `post_staging_unlink`, `pre_provenance_close`, and `post_provenance_close`. They stop the process after the staging unlink and around the provenance close at shutdown, and `FailpointRecoveryTests` (`tests/test_tod_dl.py`) covers each. E06 does not use them.
- After each failpoint fires and the process exits, restart the controller and assert idempotent reconciliation, scoped per failpoint since two different functions in `src/tod_dl.py` handle the three landing states, not one: failpoints (a) and (b) leave the row at `status='promoting'` and are picked up by `reconcile_promotions()`; failpoint (c) leaves the row already at `status='complete'` with `cleanup_completed_at` still unset (the first `complete` transition, line 1905, lands before the kill point) and is picked up by the separate `cleanup_completed_staging()` function, which finishes the staging unlink and the second `complete` transition. For failpoint (b), assert the final file is preserved, its recorded hash matches, and no duplicate completion record or duplicate final file results, per `reconcile_promotions()`'s existing handling. For failpoint (c), assert the same outcome, per `cleanup_completed_staging()`'s existing handling. For failpoint (a), where the durable `promoting` record exists but the final file was never created, extend `reconcile_promotions()` (see the resolved open question below) to recreate `os.link(staging, target)` and complete automatically; assert that outcome, not `review_required`.
- Read the existing controller finalization code (`src/tod_dl.py`) before placing the failpoints; place them at the exact three steps the parent specification names, not at steps that only approximate them.

Resolved: `reconcile_promotions()` has no code path that recognizes "digest already durably recorded in a `promoting` row, staging intact, target simply not yet created" — failpoint (a)'s exact landing state — and instead falls into the same `review_required` branch used for a genuinely corrupted or mismatched promotion. The fall-through happens at the `target_matches` check: `is_safe_final_path(target)` calls `target.lstat()`, catches the resulting `FileNotFoundError` internally, and returns `False`; `target_matches` then short-circuits to `False` before `sha256sum(target)` is ever called, so no exception propagates to `reconcile_promotions()`'s own `except OSError` handler.

Extend `reconcile_promotions()` rather than accept `review_required`. `SPEC-reliable-acquisition.md` already requires that "restart must reconcile a durable promotion intent with the existing final file rather than blindly labeling it complete or discarding its computed hash" — the digest for failpoint (a) is already durably recorded and was computed before the kill, so routing it to manual review discards a trustworthy result instead

of reconciling it. At the `target_matches` check, when `target` is absent (`is_safe_final_path` returns `False` because `target.lstat()` raises `FileNotFoundError`) but the `promoting` row carries a digest and `staging` still exists as a regular file, recreate `os.link(staging, target)` and complete the row the same way the normal `transfer()` path does, instead of falling through to `review_required`. Retain `review_required` for every other `target_matches` failure (target present but hash mismatch, staging missing, unsafe path). This decision means the piece touches `Downloader.reconcile_promotions()` itself, not only test infrastructure; the Deliverables section below reflects that.

4. One-million-row queue generator and resource sampler (for E10)

Requirement: "One million synthetic queue rows... Meet the resource and responsiveness targets... do not request every URL," together with the engineering targets: combined peak RSS below 512 MiB, at most 64 admitted unfinished jobs with at most 4 actively transferring, five-second idle-slot refill, two-second status latency, 30-second graceful shutdown. (parent specification, scenario table and "Initial acceptance targets.")

- Add a generator that produces one million syntactic queue rows (URLs and expected metadata) without writing one million fixture bodies; the fixture server may synthesize response bodies for these rows on demand, using the same deterministic generator as smaller fixtures, sized small enough that a full transfer of every row is not the point of this scenario.
- Add a sampler that records controller and every child engine process RSS, summed, at a fixed interval (document the interval; the parent specification requires it to be documented, not a specific value) for the duration of the run, and reports the maximum sum. Walk the full process tree rooted at the controller, not only its direct children: the controller spawns `torsocks -i aria2c` per engine worker (`Downloader.run_aria2()` , `src/tod_dl.py`), and the transferring `aria2c` process may be a grandchild rather than a direct child, depending on whether `torsocks` exec-replaces itself on the evaluation machine. Confirm which it is during implementation instead of assuming direct-children-only is sufficient.
- Assert, from the fixture server's request log, that the number of distinct URLs requested stays within the configured admission bound across the run, not one million — this is the scenario's core assertion, not an optimization detail. Candidate 1 (one aria2 process per URL, evaluated first per the parent specification's "Candidates and decision order") has no separate admission stage: `Downloader.run()` (`src/tod_dl.py`) admits a row only immediately before submitting it to the worker pool, so "admitted" and "actively transferring" are the same count, bounded by the single `--workers` value. The parent specification's "at most 64 admitted, unfinished jobs" and "at most 4 actively transferring" targets are therefore not two independently checkable bounds under candidate 1; assert only that requested-URL count stays within `--workers` , and record the 64-job ceiling as not applicable to this candidate rather than asserting it as a separate, currently nonexistent bound. If a later candidate (for example, the bounded-RPC-feed candidate) introduces a real admission queue distinct from active transfers, this bullet must be revised to assert both bounds independently.

- Measure status-request latency by issuing a local status request at intervals during the run and recording response time; assert the two-second target.
- Measure shutdown time by sending the graceful-shutdown signal near the end of the run and recording time to process exit; assert the 30-second target, including the child-termination escalation path if the controller does not exit gracefully within it.

Open question: decide where the one-million-row queue file lives during the run (generated fresh per run vs. cached). Generate it fresh in the isolated run directory each time, consistent with every other scenario's isolation requirement, unless generation time itself becomes a measured confound.

5. Concurrency-timing assertions (for E13)

Requirement: "Slow large file alongside small files and a due retry. Refill idle slots promptly; the large file does not block unrelated work." Engineering target: "Refill an eligible idle slot within five seconds, excluding deliberate staggering, cooldown, storage pause, and validation backpressure." (parent specification, scenario table and "Initial acceptance targets.")

- Add a scripted fixture mix: one slow large file (throttled via the existing scripted-response delay mechanism, not a separate throttling implementation), several small files that complete quickly, and one item already in `retry_wait` with a due retry timestamp at run start.
- Record the timestamp of every completion and every new admission from the controller's durable state: the `download_transitions` table already records a `recorded_at` timestamp on every status change, including the `active` transition (admission) and the `complete` transition (completion). The fixture server's event log cannot supply these timestamps; its recorded events carry no timestamp field (`FixtureServer._record`, `src/evaluation/acquisition_evaluation.py`).
- Assert that each idle-slot refill following a completion or a due retry occurs within five seconds of that completion or due timestamp, excluding any interval attributable to deliberate staggering, cooldown, storage pause, or validation backpressure that the run's configuration explicitly set — record which exclusion, if any, applied to each measured interval.
- Assert that the slow large file's transfer does not delay the small files' admission or completion: compare the small files' completion timestamps against a run of the same small files with no concurrent large file, or against the five-second refill target directly, whichever the implementation finds simpler to make objectively checkable.

6. Five-item scoped run assertion (for E11)

Requirement: "Five-item run selected from a larger queue... Only those five distinct items may transfer, including after retries and restart." (parent specification, scenario table.)

- Build a queue file with more than five distinct URLs (for example, ten). Drive the run with the controller's existing `--max-files 5` flag rather than a new selection mechanism: `Downloader.scope_run()` (`src/tod_dl.py`) already populates `run_items` with the first five queue-order rows whose target does not already exist, and every later admission and retry path joins against `run_items`. No controller-code change is needed for the boundary itself; this piece is a harness invocation and an assertion, not new controller logic.
- Pass the same explicit `--run-id` value on the initial run and on the restart. `scope_run()` is idempotent only when `run_id` matches an existing `run_items` population: it returns early when `previous` (the existing row count for that `run_id`) is nonzero. Without an explicit `--run-id`, the controller derives a new timestamp-based run ID on restart, which would let `scope_run()` reselect up to five different rows from the larger queue instead of resuming the same five. Confirm this during implementation before writing the assertion; a harness that omits `--run-id` would silently test the wrong thing.
- Include, among the five selected items, one row already in `retry_wait` with a due retry at run start, so the requirement's "including after retries" clause is exercised, not only inferred from the restart case.
- Restart the controller once, against the same isolated destination and state directories, the same queue file, and the same `--run-id`, after the five selected items reach a terminal or in-progress state.
- Assert, from the fixture server's request log, that the set of distinct URLs requested is exactly the five selected URLs, both before and after the restart — never a sixth URL from the larger queue file, and never fewer than five once the run completes.
- Assert, from durable state, that `run_items` for this `run_id` contains exactly the same five URLs before and after the restart, in the same `queue_rank` order.

Resolved: a `downloads` row exists for every queue URL, including the sixth-and-later rows never selected into `run_items`. `Downloader.run()` calls `import_queues(db)` unconditionally before `scope_run()` (`src/tod_dl.py`), and `import_queues()` inserts a row for every URL read from the queue file (`INSERT OR IGNORE INTO downloads ...`) regardless of whether `scope_run()` later selects it. The sixth-and-later rows therefore do have a `downloads` row; `scope_run()` only decides which URLs get a `run_items` row and are ever transferred. Assert that the sixth-and-later URLs are never requested and never gain a `run_items` row, not that no `downloads` row exists for them.

Deliverables

Extend `src/evaluation/acquisition_evaluation.py`, `src/evaluation/aria2_evaluation_adapter.py`, `src/evaluation/run_acquisition_evaluation.py`, and the controller finalization path in `src/tod_dl.py` to add the six pieces above. In `src/tod_dl.py`, this means the three failpoints (Section 3) and the `reconcile_promotions()` extension for failpoint (a) resolved in that section's open question; it does not mean a fix for the E02 `is_incomplete_body_failure()` gap (Section 1), which stays out of scope for this specification. Section 6 adds no controller code, only a scenario function (`e11_five_item_scoped_run`, for example) in `src/evaluation/run_acquisition_evaluation.py` and its harness-side assertions. Add

corresponding entries to `tests/test_acquisition_evaluation.py` for harness behavior that unit tests can cover (generator determinism, sampler interval correctness, failpoint gating) separately from the end-to-end scenario runs that only `src/evaluation/run_acquisition_evaluation.py` can produce. Do not run E02, E05, E06, E10, E11, or E13 as part of this specification; running them and recording results belongs to a later evaluation report, per [the acquisition tool evaluation specification](#).

Next steps

All six pieces are built. Run E02, E05, E06, E10, E11, and E13 and record results in a later evaluation report, per [the acquisition tool evaluation specification](#).

Specification: acquisition fault-recovery tests

Status: implemented, September 20, 2026. The test suite covers every recovery case and scheduler condition. This specification defines the fault-test contract. It supplements the [reliable acquisition specification](#). Tests must protect acquired data and must not make source requests.

Test boundary

Fault tests must use a temporary destination, temporary state directory, and a local fake transfer engine. Tests must not read, write, rename, or hash files below `downloaded_files/` , `archive/` , `pst_extracted/` , `msg_extracted/` , or `output/` .

The fake engine must simulate partial files, successful completion, terminal failure, a stalled transfer, and a changed remote representation. The harness must expose controlled failure points in SQLite updates, file flushes, exclusive final-file creation, staging cleanup, and controller shutdown.

Required recovery cases

Each test must stop the controller at one named failure point, start a new controller with the same run ID and unchanged queue input, then check durable state and filesystem results.

The standard downloader test file must name recovery tests by the failure point. Each test must build a new temporary root. The test must use fixture bytes and a local fake transfer engine. The test must not start `aria2c` , `torsocks` , or a source request.

Failure point	Required result after restart
Before attempt state commits	The item remains eligible or is safely requeued.
During an active partial transfer	The partial and its control metadata remain available for resume.
After engine success but before hashing completes	The item is not final. The controller rechecks staging.

Failure point	Required result after restart
After hash and promotion intent commit	The controller reconciles the intended final path and digest.
After exclusive final creation but before completion commit	The controller hashes the existing final and records completion only on a match.
After completion commit but before staging cleanup	The final remains complete and cleanup is safe to repeat.
During candidate creation	The existing final remains unchanged and the incoming bytes remain reviewable.
During SQLite commit or WAL checkpoint	The controller reports a local failure and does not admit new work until recovery.

Transfer and scheduler cases

The harness must test transfer failures without network access. It must verify that a retry does not admit a queue item outside the immutable selection.

The harness must test these scheduler conditions:

- `retry_now` wakes admission while another transfer remains active.
- An eligible retry fills an idle worker slot within one admission poll.
- A time limit stops admission only at expiry.
- A storage-reserve failure stops admission without changing final files.
- A zero-progress transfer reaches the configured stall policy without a busy loop.

Assertions and artifacts

Every fault test must assert final-file paths, file bytes, SHA-256 digests, SQLite state, transition order, and staging or candidate preservation. The test must check that no final path is overwritten. The test must check that a restart does not expand the selected item set.

The test harness must retain its temporary artifacts only when a test fails. When it retains artifacts, it must print their path and must not copy them into the repository.

Acceptance criteria

The project must run all fault tests in the standard downloader test command. The tests must pass before a downloader change is handed off. A failed fault test blocks unattended acquisition until an operator records the failure and the project resolves it.

Specification: multi-route acquisition

Status: implemented, September 23, 2026. The shared `Route` vocabulary (`src/routing.py`), the acquisition engine's per-item route resolution, the queue `route=` token, the mixed-route startup gate, and the route-aware aria2 invocation, HEAD probe, and Tor circuit-renewal gating are implemented. This specification governs how the acquisition engine picks a network route for each selected item, reusing the same vocabulary the discovery network fetch already uses for snapshot fetches.

Outcome and scope

The acquisition engine was Tor-only: every transfer, HEAD probe, and outage recovery went through `torsocks -i`, unconditionally. This specification lets an operator route a selected item through Tor, a direct connection, or a configured proxy, per item, without weakening Tor isolation for any item that still needs it. It does not add a new transport (aria2 and its existing process model are unchanged); it only makes the wrap around aria2 conditional on a resolved `Route`.

Route resolution

`src/routing.py` defines `Route` (`kind: tor, direct, or proxy; proxy_name` when `kind == "proxy"; label`, e.g. `"tor"`, `"direct"`, `"proxy:corp-relay"`) and `resolve_route(host, *, is_onion, require_tor, configured_proxy)`. An `.onion` host always resolves to `tor` and accepts no override, including a queue line that names a different route. Otherwise an explicit `tor` request or a missing route selection forces `tor`; then a configured proxy wins; a bare clearnet host resolves to `direct`.

A queue line may add a trailing `route=` token: `route=tor`, `route=direct`, or `route=proxy:<name>`. A line without the token keeps the pre-multi-route behavior exactly: every host, onion or clearnet, resolves to `tor`. This keeps every existing queue file working unchanged; a non-default route is opt-in per line. The resolved route's `label` is stored on the item's `downloads.route` column at import time and does not change afterward for that item, matching the run's other immutable selection properties.

Engine integration points

Five call sites in `src/downloader/` read the item's resolved route instead of hardcoding Tor:

- `downloader/transfer.py` 's `run_aria2()` builds its command prefix from `routing.build_route_prefix(route, torsocks_path)`: `tor` wraps aria2c in `torsocks -i`; `direct` and `proxy` add no prefix. A `proxy` route also adds aria2's `--all-proxy=<name>` argument (`routing.aria2_proxy_args(route)`). Substituting aria2's proxy option for a Tor SOCKS listener is never valid for the `tor` route; see [the evaluation specification's Tor and process model section](#) for why, which now scopes that rule to the `tor` route explicitly.
- `downloader/core.py` 's `run()` computes every selected item's resolved route before starting the transfer loop and calls `tor.verify_tor_isolation()` only if at least one resolves to `tor`. The durable manifest's `tor_isolation_preflight` key records whether the preflight ran and, when it did not, why.
- `downloader/transfer.py` 's `transfer()` requests a fresh Tor circuit (`request_newnym`) after a connectivity failure only when the failed item's route is `tor`. A `direct` or `proxy` item's connectivity failure falls straight through to the existing backoff and retry; Tor is not carrying that item's traffic, so renewing its circuits fixes nothing.
- `downloader/representation.py` 's `head_request()` (the outage-recovery and resume-representation probe) uses the same route-driven command shape as `run_aria2`.
- `downloader/state.py` 's `next_pending()` and `next_recheck()` select the `route` column alongside the item's other transfer state, so every reader of a pending or rechecked row has the route it must use.

Schema and provenance

`downloads.route` (nullable `TEXT`) holds the resolved route's `label`. `NULL` or absent means the historical, pre-migration `tor` behavior; an existing state directory opened by a build with this column adds it via a guarded `ALTER TABLE`, never rewriting existing rows' values. Provenance's `attempt_finished` event records `route` alongside its other per-attempt fields, so an evidence review can see which route served each attempt.

Mixed-route startup gate

A queue whose selected items resolve to more than one route kind (for example, one `.onion` item mixed with a clearnet `route=direct` item) aborts the run at startup, before any admission, unless the operator passes `--allow-mixed-routes`. An operator who did not expect a mixed selection must not discover after the fact that some items silently left Tor; passing the flag is an explicit, one-time acknowledgment for that run. `.onion` and `require_tor` items always resolve to `tor`, so this gate only fires when a queue actually combines a Tor item with a `direct` or `proxy` item.

Acceptance and next steps

Use synthetic fixtures and temporary state directories to demonstrate:

- A queue line without a `route=` token still resolves to `tor`, matching pre-multi-route behavior exactly.
- An `.onion` host resolves to `tor` even when its queue line names `route=direct` or `route=proxy:<name>`.
- `run_aria2()`'s and `head_request()`'s command shape omits `torsocks` for `direct` and `proxy` routes and includes it only for `tor`.
- `verify_tor_isolation()` runs when any selected item resolves to `tor` and is skipped when none does.
- A connectivity failure on a `direct` or `proxy` item never calls `request_newnym`; a `tor` item's connectivity failure still does.
- Opening a state directory created before this column existed adds `route` without altering any other row value.
- A mixed-route queue aborts at startup without `--allow-mixed-routes` and proceeds, running the Tor preflight, once the flag is passed.

No live scraping, real Tor circuit, or bulk acquisition is part of writing or reviewing this specification, per AGENTS.md's dry-run rule.

Native route selection

The native transfer engine owns URL-queue admission. Tor routes use isolated body children. Direct routes and HTTP proxies use the shared body transport. Unsupported native proxy kinds fail before contact. Descriptor routing keeps its existing support.

Contract owner: [native cutover](#).

Specification: acquisition provenance manifest

Status: implemented, September 20, 2026. The controller writes signed provenance records, and the verifier checks them. This specification defines the provenance contract. It supplements the [reliable acquisition specification](#). The SQLite database remains the controller's recovery authority.

Purpose and boundary

The downloader must produce an exportable record that links each final file to its source request, local validation, and finalization result. The record can prove that a named local file has a recorded SHA-256 digest. The record cannot prove that the source served an original or complete collection.

The controller must not modify `downloaded_files/` to add provenance data. It must write each session record set only below `download-state/provenance/<run-id>/<session-id>/`. The monitor must remain read-only and must not create provenance records.

Record set

For each controller session, the controller must create one new record directory and write these files:

- `events.jsonl` contains append-only provenance events.
- `summary.json` contains the final session summary and the final event digest.
- `schema.json` contains the versioned JSON Schema and canonicalization method.

The controller must create each record directory with mode `0700`. It must create each record file with mode `0600`. The controller must create and fsync the record directory before it writes a record. When it writes `summary.json` or `schema.json`, it must fsync a temporary file, atomically replace the target, and fsync the record directory.

`events.jsonl` must contain one UTF-8 JSON object and one LF byte per line. Before the controller records the related durable completion state, it must flush and fsync the event file. A failed record write must stop new admission and report a local failure. A completed file without a durable provenance event is unresolved.

Event format

Each event must contain these fields. Unknown source metadata must use JSON `null`. Timestamps must use UTC RFC 3339 format with a `Z` offset.

Field	Requirement
<code>schema_version</code>	Positive integer schema version.
<code>event_id</code>	Unique UUID for the event.
<code>sequence</code>	Positive integer that increases by one within the record set.
<code>event_type</code>	One of <code>run_started</code> , <code>attempt_finished</code> , <code>finalized</code> , <code>candidate_created</code> , <code>run_closed</code> , or <code>local_failure</code> .
<code>run_id</code> , <code>session_id</code>	Immutable selected run identifier and controller-session identifier.
<code>recorded_at</code>	Event creation time.
<code>previous_digest</code>	SHA-256 digest of the preceding event, or <code>null</code> for sequence one.
<code>event_digest</code>	SHA-256 digest of the canonical event object without <code>event_digest</code> .

The controller must serialize event objects with the JSON Canonicalization Scheme (JCS), RFC 8785.

`schema.json` must be a JSON Schema Draft 2020-12 document encoded as UTF-8. It must identify `JCS-RFC8785`, define both event and summary objects, require the fields and values in this specification, and reject unknown fields for its schema version. `schema_sha256` is the SHA-256 digest of the exact UTF-8 bytes of `schema.json`. The controller must calculate each event digest from UTF-8 canonical bytes. Each digest must be a lowercase, 64-character hexadecimal SHA-256 value. Consumers must reject an invalid JSON object, duplicate JSON object keys, a missing required field, an unknown event type, a sequence gap, a duplicate event ID, a changed previous digest, or an invalid event digest.

For sequence one, `previous_digest` must be `null`. For every later sequence, `previous_digest` must be a digest in the format above. Every event `run_id` and `session_id` must equal the identifiers in its record-directory path.

Acquisition fields

Every item event, which is `attempt_finished` , `finalized` , or `candidate_created` , must include `item_id` . The value must be the stable item ID from the immutable selected manifest. This field links events for one item.

`attempt_finished` must include these fields. The controller must include every field, even when its value is `null` .

Field	Type and requirement
<code>item_id</code>	Stable selected-manifest item ID.
<code>source_url</code>	Exact requested URL.
<code>attempt_number</code>	Positive integer.
<code>request_started_at</code> , <code>request_finished_at</code>	UTC RFC 3339 timestamps.
<code>final_url</code>	Final URL string, or <code>null</code> when no response URL exists.
<code>redirect_chain</code>	Array of zero or more URL strings.
<code>http_status</code>	Integer from 100 through 599, or <code>null</code> .
<code>outcome</code>	One of <code>success</code> , <code>retryable_failure</code> , <code>unavailable</code> , <code>access_denied</code> , <code>validation_failed</code> , or <code>stopped</code> .
<code>response_content_length</code> , <code>response_content_range</code> , <code>response_content_type</code> , <code>response_etag</code> , <code>response_last_modified</code>	Raw response-field string, or <code>null</code> .
<code>route</code>	The resolved <code>Route.label</code> this attempt used (<code>tor</code> , <code>direct</code> , or <code>proxy:<name></code>), or <code>null</code> for an attempt recorded before this field existed. See the multi-route acquisition specification .

`redirect_chain` lists the URLs between `source_url` and `final_url` , in order. It is empty when the request had no redirect. The controller takes the response fields from the last HTTP response of the attempt. If a redirect URL carries user-info credentials, `final_url` is `null` and `redirect_chain` is empty.

`access_denied` records an HTTP 401 or 403 response.

The record must never include request cookies, authorization values, Tor control cookies, RPC secrets, or monitor capability tokens. The controller must reject a source or redirect URL with user-info credentials or a query string. The controller cannot tell which query values grant access, so it treats every query string as

access-granting. Queue import rejects such a source URL. If any URL of an attempt has user-info credentials or a query string, `final_url` is `null` and `redirect_chain` is empty. The event writer rejects any event URL with either. The controller must record an unavailable or redacted value explicitly rather than silently omit a required field.

`finalized` must include `item_id`, `source_url`, `logical_relative_path`, `final_relative_path`, `byte_count`, `sha256`, `validation_method_version`, and `finalized_at`. `byte_count` must be a nonnegative integer. `sha256` must use the digest format above. Both relative paths must use slash-separated relative paths and must not contain `.` or `..` path components. `finalized_at` must be a UTC RFC 3339 timestamp.

`finalized` must also include an `expected_size`, `expected_checksum`, `etag`, and `last_modified` validation object. Each object must contain `available` and `compared` Boolean fields and a `value` field. The value is `null` when `available` is false. `compared` must be false when `available` is false. When available, `expected_size.value` is a nonnegative integer. `expected_checksum.value`, `etag.value`, and `last_modified.value` are strings.

The controller sets `etag.value` and `last_modified.value` from the values stored for resume checks. If none are stored, it uses the response fields of the final attempt. `etag.compared` is true only when a resume check confirmed the stored ETag against the source. `expected_size.value` is the exact full size that the final response announced: `Content-Length` of a 200 response, or the total in `Content-Range` of a 206 response. An inventory size token is never an expected size. `expected_size.compared` is true when the controller compared this value with the staged byte count before promotion. A mismatch blocks promotion: the item moves to `review_required` with review code `size_mismatch`, and the candidate reason is `validation_failed`. `last_modified.compared` stays false until the controller compares it.

`candidate_created` must include `item_id`, `staging_relative_path`, `candidate_relative_path`, `sha256`, and `finalization_failure_reason`. `sha256` is `null` when hashing did not complete. The failure reason must be one of `validation_failed`, `destination_collision`, `unsafe_path`, `promotion_error`, `recovery_review`, or `body_integrity_replaced`. `body_integrity_replaced` applies only to an r5 body-integrity replacement. The controller records `recovery_review` for a changed remote representation, for missing version protection, and when promotion recovery cannot finish and the staged file is a safe regular file. In that last case the staged file stays in place. It must not label a candidate as evidence final. The controller records `unsafe_path` when the destination path holds a symlink or a non-directory parent. It records `promotion_error` when creating the final link fails with an operating-system error, during a live transfer or during promotion recovery. For these two reasons the staged file stays in place, because safe-path remediation reads it there. The candidate path is a hard link to the staged file. If the controller cannot create that link, the item still moves to `review_required`, and the controller writes no `candidate_created` event for it. When safe-path remediation later promotes the staged file, the controller removes each candidate link that shares the staged file's inode. `staging_relative_path` is relative to `download-state/`. `candidate_relative_path` is relative to `download-state/redownload-candidates/`.

`run_started` must include `queue_input_digests`, `selection_settings`, and `selected_item_count`, which is a nonnegative integer. `run_closed` must include `closed_at`, `close_reason`, and `durable_outcome_counts`. `close_reason` must be one of `finished`, `stopped`, `time_limit`, `local_failure`, or `interrupted`. Every durable-outcome count must be a nonnegative integer. `local_failure` must include `failure_code` and a redacted `failure_detail`. `failure_code` must be one of `record_write`, `database_write`, `permission`, `storage`, or `other_local_failure`.

Summary and verification

`summary.json` must include the run ID, session ID, queue-input digests, selection settings, event count, final event digest, selected-item count, durable outcome counts, the session finalized count, the SHA-256 digest of `schema.json`, and an Ed25519 signature. The durable outcome counts cover all items of the run. `session_finalized_count` is the number of `finalized` events that this session wrote. The verifier must require that it equals the number of `finalized` events in the session and does not exceed the durable `complete` count. A resumed session can finalize fewer files than the run total. The signature must cover the JCS canonical summary object without the signature field. The summary must also identify the signing public-key fingerprint. `schema_sha256` must use the digest format above. `signature` must be an unpadded base64url encoding of the 64-byte Ed25519 signature. `signing_key_fingerprint` must be the lowercase hexadecimal SHA-256 digest of the 32-byte Ed25519 public key. It must not contain full response bodies or copied source files.

The operator must retain the trusted Ed25519 public key or its expected fingerprint outside the record directory. A public key copied only with the record set is not a trust anchor.

The project must provide a read-only verifier. The verifier must require a trusted public key or expected public-key fingerprint. It must check the schema digest, summary signature, event sequence, digest chain, summary final-event digest, final-file existence, final-file SHA-256 digests, and byte counts. The verifier must validate the summary and every event against `schema.json`. It must verify that event and summary identifiers equal their record-directory path. The verifier must report each mismatch and return a nonzero exit status. The verifier must never repair, rename, or delete evidence files.

Acceptance criteria

Implementation is complete when tests show that a completed fixture creates a valid record set and that each of these changes causes verification failure:

- A changed final file.
- A removed or reordered event line.
- A changed event field.

- A wrong summary final-event digest.
- A changed summary signature or schema file.
- A removed final event line with a replaced matching summary.
- A `finalized` event count that differs from `session_finalized_count`.
- A missing final file.